

Digital Nurture 5.0 - Java FSE React (Week-1 Work)

PL/SQL Programming:

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Solution:

CODE:

```
/* EXERCISE 1 : CONTROL STRUCTURES */
```

```
/*TABLE CREATION*/
```

```
CREATE TABLE Customers (  
  CustomerID NUMBER PRIMARY KEY,  
  Name VARCHAR2(100),  
  DOB DATE,  
  Balance NUMBER,  
  LastModified DATE  
);
```

```
CREATE TABLE Accounts (  
  AccountID NUMBER PRIMARY KEY,  
  CustomerID NUMBER,  
  AccountType VARCHAR2(20),  
  Balance NUMBER,  
  LastModified DATE,  
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Transactions (  
    TransactionID NUMBER PRIMARY KEY,  
    AccountID NUMBER,  
    TransactionDate DATE,  
    Amount NUMBER,  
    TransactionType VARCHAR2(10),  
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)  
);
```

```
CREATE TABLE Loans (  
    LoanID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    LoanAmount NUMBER,  
    InterestRate NUMBER,  
    StartDate DATE,  
    EndDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    Position VARCHAR2(50),  
    Salary NUMBER,  
    Department VARCHAR2(50),  
    HireDate DATE  
);
```

```
/* SAMPLE DATA INSERTION */
```

```
INSERT INTO Customers  
VALUES (1, 'John Doe',  
        TO_DATE('1985-05-15','YYYY-MM-DD'),  
        1000, SYSDATE);
```

```
INSERT INTO Customers  
VALUES (2, 'Jane Smith',  
        TO_DATE('1990-07-20','YYYY-MM-DD'),  
        1500, SYSDATE);
```

```
INSERT INTO Accounts  
VALUES (1, 1, 'Savings',  
        1000, SYSDATE);
```

```
INSERT INTO Accounts  
VALUES (2, 2, 'Checking',  
        1500, SYSDATE);
```

```
INSERT INTO Transactions
VALUES (1, 1, SYSDATE,
       200, 'Deposit');
```

```
INSERT INTO Transactions
VALUES (2, 2, SYSDATE,
       300, 'Withdrawal');
```

```
INSERT INTO Loans
VALUES (1, 1, 5000, 5,
       SYSDATE,
       ADD_MONTHS(SYSDATE,60));
```

```
INSERT INTO Employees
VALUES (1, 'Alice Johnson',
       'Manager',
       70000,
       'HR',
       TO_DATE('2015-06-15','YYYY-MM-DD'));
```

```
INSERT INTO Employees
VALUES (2, 'Bob Brown',
       'Developer',
       60000,
       'IT',
       TO_DATE('2017-03-20','YYYY-MM-DD'));
```

```
/* EXTRA DATA */
```

```
INSERT INTO Customers
VALUES (3, 'Robert Wilson',
       TO_DATE('1955-08-10','YYYY-MM-DD'),
       15000,
       SYSDATE);
```

```
INSERT INTO Customers
VALUES (4, 'Mary Johnson',
       TO_DATE('1960-02-15','YYYY-MM-DD'),
       12000,
       SYSDATE);
```

```
INSERT INTO Loans
VALUES (2, 3, 10000, 7,
       SYSDATE,
       SYSDATE + 20);
```

```
INSERT INTO Loans
VALUES (3, 4, 15000, 8,
```

```

        SYSDATE,
        SYSDATE + 25);

COMMIT;

/* SCENARIO 1 - Interest Rate Discount */

DECLARE
    v_age NUMBER;
BEGIN

    FOR cust IN (
        SELECT CustomerID, DOB
        FROM Customers
    )
    LOOP

        v_age := TRUNC(
            MONTHS_BETWEEN(
                SYSDATE,
                cust.DOB) / 12);

        IF v_age > 60 THEN

            UPDATE Loans
            SET InterestRate =
                InterestRate - 1
            WHERE CustomerID =
                cust.CustomerID;

            DBMS_OUTPUT.PUT_LINE(
                'Discount applied for Customer ID: '
                || cust.CustomerID);

        END IF;

    END LOOP;

    COMMIT;

END;
/

/* SCENARIO 2 - VIP Customers */

ALTER TABLE Customers
ADD IsVIP VARCHAR2(5);

```

BEGIN

```
FOR cust IN (
    SELECT CustomerID,
           Balance
    FROM Customers
)
LOOP
```

IF cust.Balance > 10000 THEN

```
UPDATE Customers
SET IsVIP = 'TRUE'
WHERE CustomerID =
    cust.CustomerID;
```

```
DBMS_OUTPUT.PUT_LINE(
    'VIP Status Assigned to Customer ID: '
    || cust.CustomerID);
```

END IF;

```
END LOOP;
```

COMMIT;

END;
/

```
/* SCENARIO 3 - Loan Reminder */
```

BEGIN

```
FOR loan_rec IN
(
    SELECT c.Name,
           l.LoanID,
           l.EndDate
    FROM Customers c
    JOIN Loans l
    ON c.CustomerID = l.CustomerID
    WHERE l.EndDate
           BETWEEN SYSDATE
           AND SYSDATE + 30
)
LOOP
```

DBMS_OUTPUT.PUT_LINE(

```

        'Reminder: Dear '
        || loan_rec.Name
        || ', your Loan ID '
        || loan_rec.LoanID
        || ' is due on '
        || TO_CHAR(
            loan_rec.EndDate,
            'DD-MON-YYYY'
        )
    );

```

```
END LOOP;
```

```
END;
```

```
/
```

OUTPUT:

The screenshot shows a SQL IDE window with a dark theme. The 'Script output' tab is active. At the top, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. Below the tabs, there are icons for a trash can and a download arrow. The main area displays the following SQL commands and their outputs:

```

SQL> CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    DOB DATE,...
Show more...

Table CUSTOMERS created.

Elapsed: 00:00:00.023

SQL> CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),...
Show more...

```

This screenshot continues the output from the previous one, showing the creation of the Transactions and Loans tables. The SQL commands and their outputs are as follows:

```

Table ACCOUNTS created.

Elapsed: 00:00:00.020

SQL> CREATE TABLE Transactions (
    TransactionID NUMBER PRIMARY KEY,
    AccountID NUMBER,
    TransactionDate DATE,...
Show more...

Table TRANSACTIONS created.

Elapsed: 00:00:00.019

SQL> CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    LoanAmount NUMBER,...

```

```
CUSTOMERID NUMBER,  
LoanAmount NUMBER,...  
Show more...
```

Table LOANS created.

Elapsed: 00:00:00.019

```
SQL> CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    Position VARCHAR2(50),...  
Show more...
```

Table EMPLOYEES created.

Elapsed: 00:00:00.014

```
SQL> INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)  
VALUES (1, 'Alice Johnson', 'Manager', 70000, 'HR',  
        TO_DATE('2015-06-15', 'YYYY-MM-DD'))
```

1 row inserted.

Elapsed: 00:00:00.015

```
SQL> INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)  
VALUES (2, 'Bob Brown', 'Developer', 60000, 'IT',  
        TO_DATE('2017-03-20', 'YYYY-MM-DD'))
```

1 row inserted.

Elapsed: 00:00:00.003

```
SYSDATE,  
SYSDATE + 20)
```

1 row inserted.

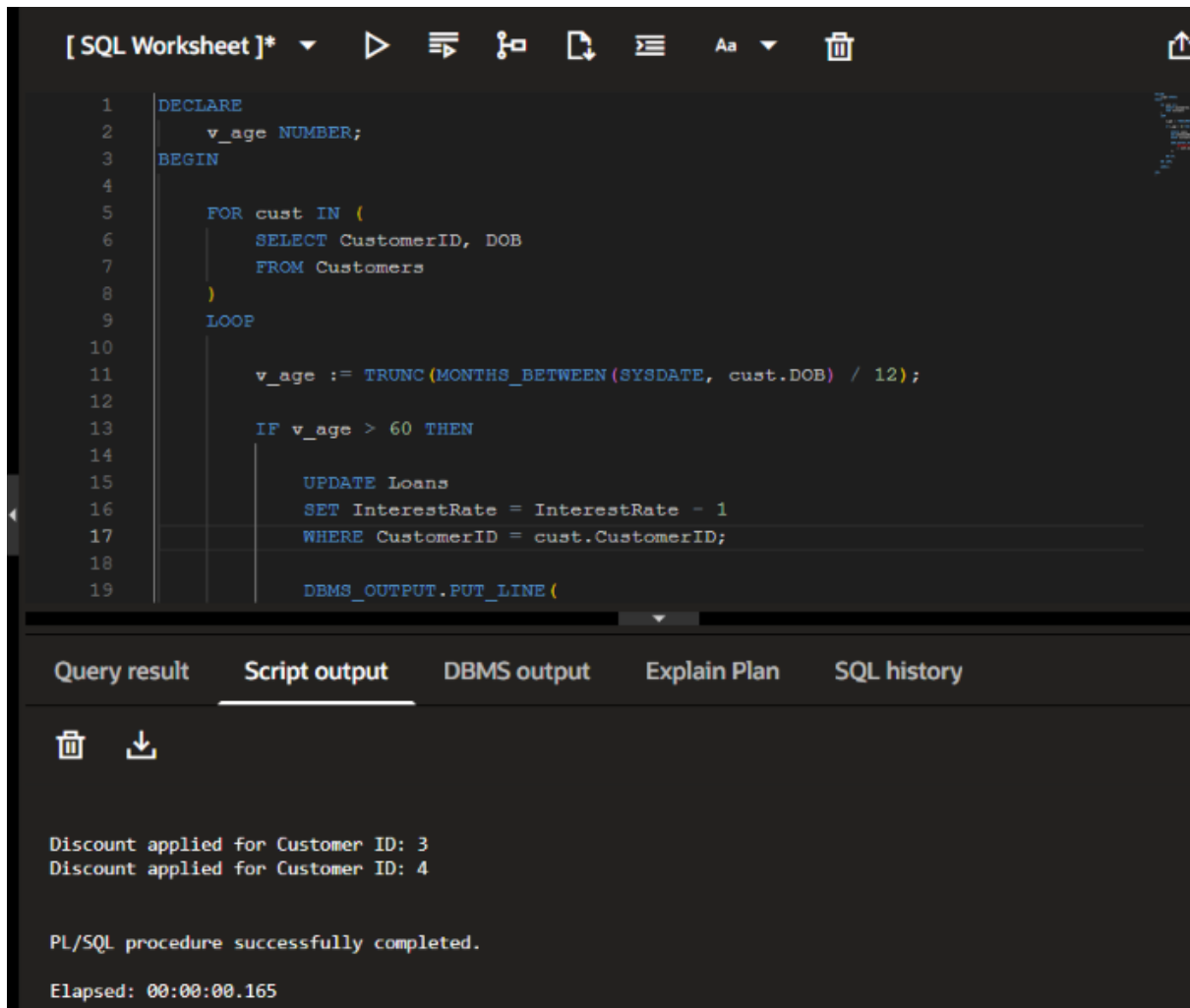
Elapsed: 00:00:00.006

```
SQL> INSERT INTO Loans  
VALUES (3, 4, 15000, 8,  
        SYSDATE,  
        SYSDATE + 25)
```

1 row inserted.

Elapsed: 00:00:00.003

Scenario 1:



The screenshot shows a SQL Worksheet interface with a PL/SQL procedure script in the editor. The script declares a variable `v_age` of type `NUMBER`, loops through customers, calculates their age, and updates the interest rate for loans of customers aged 60 or older. The output pane shows the results of the script execution.

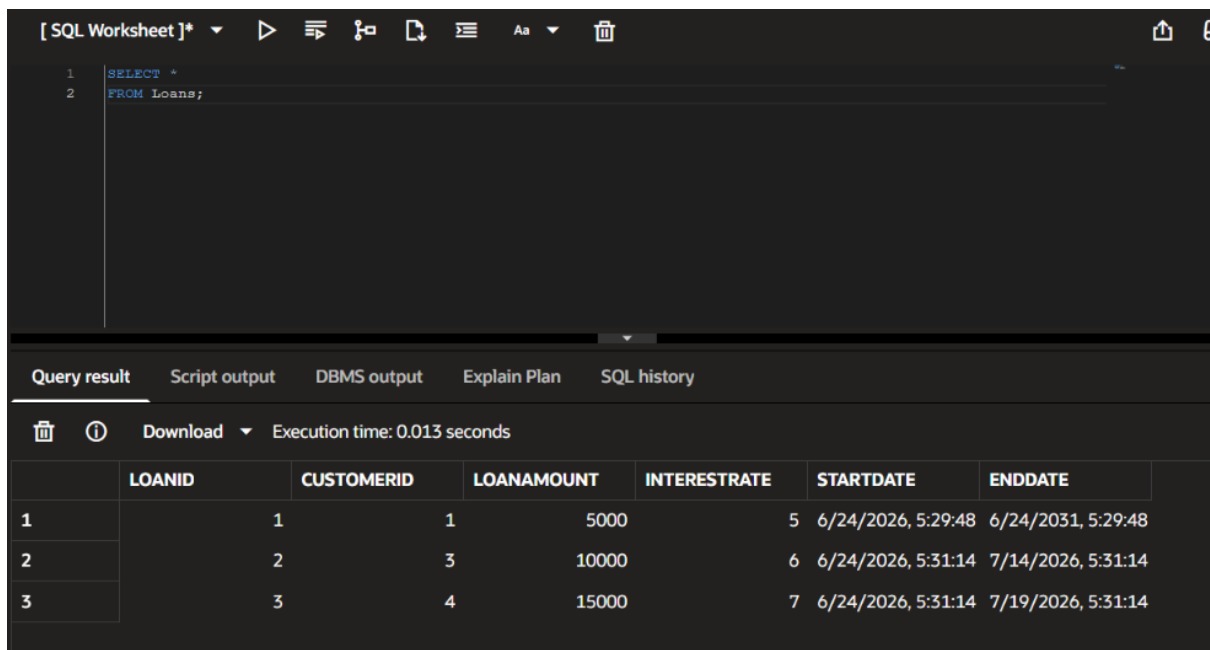
```
1 DECLARE
2     v_age NUMBER;
3 BEGIN
4
5     FOR cust IN (
6         SELECT CustomerID, DOB
7         FROM Customers
8     )
9     LOOP
10
11         v_age := TRUNC(MONTHS_BETWEEN(SYSDATE, cust.DOB) / 12);
12
13         IF v_age > 60 THEN
14
15             UPDATE Loans
16             SET InterestRate = InterestRate - 1
17             WHERE CustomerID = cust.CustomerID;
18
19             DBMS_OUTPUT.PUT_LINE (
```

Query result | **Script output** | DBMS output | Explain Plan | SQL history

Discount applied for Customer ID: 3
Discount applied for Customer ID: 4

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.165



The screenshot shows a SQL Worksheet interface with a simple SQL query in the editor. The query selects all columns from the `Loans` table. The output pane shows the results of the query execution as a table.

```
1 SELECT *
2 FROM Loans;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download Execution time: 0.013 seconds

	LOANID	CUSTOMERID	LOANAMOUNT	INTERESTRATE	STARTDATE	ENDDATE
1	1	1	5000	5	6/24/2026, 5:29:48	6/24/2031, 5:29:48
2	2	2	10000	6	6/24/2026, 5:31:14	7/14/2026, 5:31:14
3	3	3	15000	7	6/24/2026, 5:31:14	7/19/2026, 5:31:14

Scenario 2:

[SQL Worksheet]* Aa

```
1 ALTER TABLE Customers
2 ADD IsVIP VARCHAR2(5);
```

Query result **Script output** DBMS output Explain Plan SQL history

Elapsed: 00:00:00.165

SQL> ALTER TABLE Customers
ADD IsVIP VARCHAR2(5)

Table CUSTOMERS altered.

Elapsed: 00:00:00.038

Navigator Files

My Schema

Tables

Search objects

- ▶ ACCOUNTS
- ▶ CUSTOMERS
- ▶ EMPLOYEES
- ▶ LOANS
- ▶ TRANSACTIONS

[SQL Worksheet]* Aa

```
1 BEGIN
2
3   FOR cust IN (
4     SELECT CustomerID, Balance
5     FROM Customers
6   )
7   LOOP
8
9     IF cust.Balance > 10000 THEN
10
11       UPDATE Customers
12       SET IsVIP = 'TRUE'
13       WHERE CustomerID = cust.CustomerID;
14
15       DBMS_OUTPUT.PUT_LINE(
16         'VIP Status Assigned to Customer ID: '
17         || cust.CustomerID
18       );
19
20     END IF;
21   END LOOP;
```

Query result **Script output** DBMS output Explain Plan SQL history

VIP Status Assigned to Customer ID: 3
VIP Status Assigned to Customer ID: 4

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.159

[SQL Worksheet]*



Aa



```
1 SELECT CustomerID,  
2     Name,  
3     Balance,  
4     IsVIP  
5 FROM Customers;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

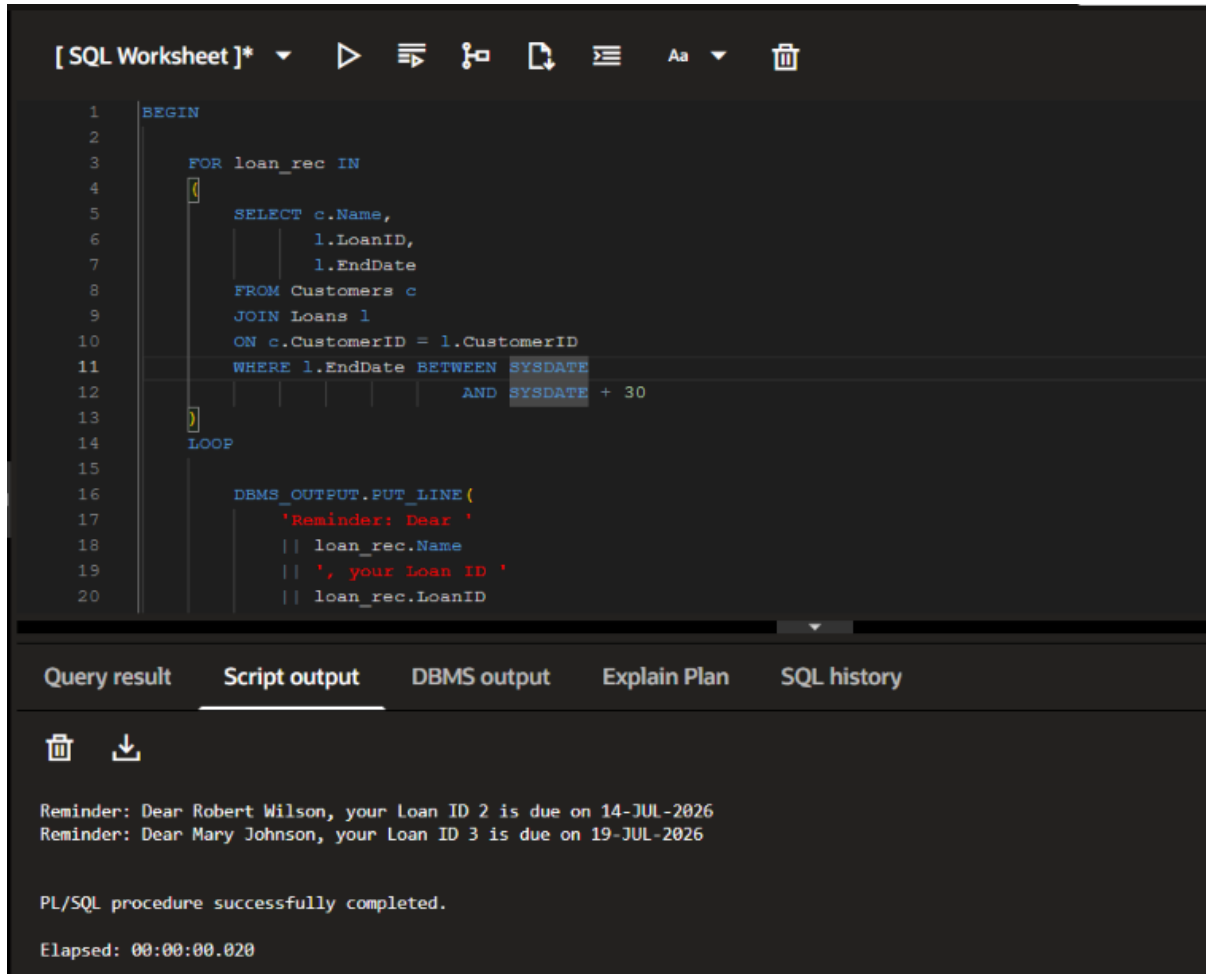


Download

Execution time: 0.008 seconds

	CUSTOMERID	NAME	BALANCE	ISVIP
1	1	John Doe	1000	(null)
2	2	Jane Smith	1500	(null)
3	3	Robert Wilson	15000	TRUE
4	4	Mary Johnson	12000	TRUE

Scenario 3:



The screenshot shows an SQL Worksheet interface with a dark theme. The top toolbar includes icons for running, saving, and other functions. The main area contains a PL/SQL script with line numbers 1 through 20. The script is a loop that iterates over loans due within the next 30 days and sends reminders. Below the script, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Script output' tab is active, showing the output of the script execution, which includes two reminder messages and a completion status.

```
1 BEGIN
2
3   FOR loan_rec IN
4   (
5     SELECT c.Name,
6           l.LoanID,
7           l.EndDate
8     FROM Customers c
9     JOIN Loans l
10    ON c.CustomerID = l.CustomerID
11   WHERE l.EndDate BETWEEN SYSDATE
12         AND SYSDATE + 30
13   )
14   LOOP
15
16     DBMS_OUTPUT.PUT_LINE(
17       'Reminder: Dear '
18       || loan_rec.Name
19       || ', your Loan ID '
20       || loan_rec.LoanID
```

Query result Script output DBMS output Explain Plan SQL history

Reminder: Dear Robert Wilson, your Loan ID 2 is due on 14-JUL-2026
Reminder: Dear Mary Johnson, your Loan ID 3 is due on 19-JUL-2026

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.020

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

- **Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

Scenario 2: Manage errors when updating employee salaries.

- **Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

Scenario 3: Ensure data integrity when adding a new customer.

- **Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

Solution:

CODE:

```
/* EXERCISE 2 : ERROR HANDLING */
```

```
/* SCENARIO 1 - SafeTransferFunds Procedure */
```

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(  
    p_fromAccount IN NUMBER,  
    p_toAccount   IN NUMBER,  
    p_amount      IN NUMBER  
)
```

```
IS
```

```
    v_balance NUMBER;
```

```
BEGIN
```

```
    SELECT Balance  
    INTO v_balance  
    FROM Accounts  
    WHERE AccountID = p_fromAccount;
```

```
    IF v_balance < p_amount THEN  
        RAISE_APPLICATION_ERROR(  
            -20001,  
            'Insufficient Funds'  
        );  
    END IF;
```

```
    UPDATE Accounts  
    SET Balance = Balance - p_amount  
    WHERE AccountID = p_fromAccount;
```

```
    UPDATE Accounts  
    SET Balance = Balance + p_amount  
    WHERE AccountID = p_toAccount;
```

```
    COMMIT;
```

```
    DBMS_OUTPUT.PUT_LINE(  
        'Funds transferred successfully.'  
    );
```

```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```

ROLLBACK;

DBMS_OUTPUT.PUT_LINE(
    'Transfer Failed: ' || SQLERRM
);

END;
/

/* Test Case */

BEGIN
    SafeTransferFunds(1, 2, 500);
END;
/

/* SCENARIO 2 - UpdateSalary Procedure */

CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_employeeID IN NUMBER,
    p_percentage IN NUMBER
)
IS
    v_count NUMBER;
BEGIN

    SELECT COUNT(*)
    INTO v_count
    FROM Employees
    WHERE EmployeeID = p_employeeID;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20002,
            'Employee ID Not Found'
        );
    END IF;

    UPDATE Employees
    SET Salary = Salary +
        (Salary * p_percentage / 100)
    WHERE EmployeeID = p_employeeID;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE(
        'Salary Updated Successfully.'
    );

```

```

);

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );

END;
/

/* Test Case */

BEGIN
    UpdateSalary(1, 10);
END;
/

/* SCENARIO 3 - AddNewCustomer Procedure */

CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_customerID IN NUMBER,
    p_name      IN VARCHAR2,
    p_dob      IN DATE,
    p_balance  IN NUMBER
)
IS
BEGIN

    INSERT INTO Customers(
        CustomerID,
        Name,
        DOB,
        Balance,
        LastModified
    )
    VALUES(
        p_customerID,
        p_name,
        p_dob,
        p_balance,
        SYSDATE
    );

    COMMIT;

```

```

    DBMS_OUTPUT.PUT_LINE(
        'Customer Added Successfully.'
    );

EXCEPTION

    WHEN DUP_VAL_ON_INDEX THEN

        DBMS_OUTPUT.PUT_LINE(
            'Error: Customer ID Already Exists.'
        );

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );

END;
/

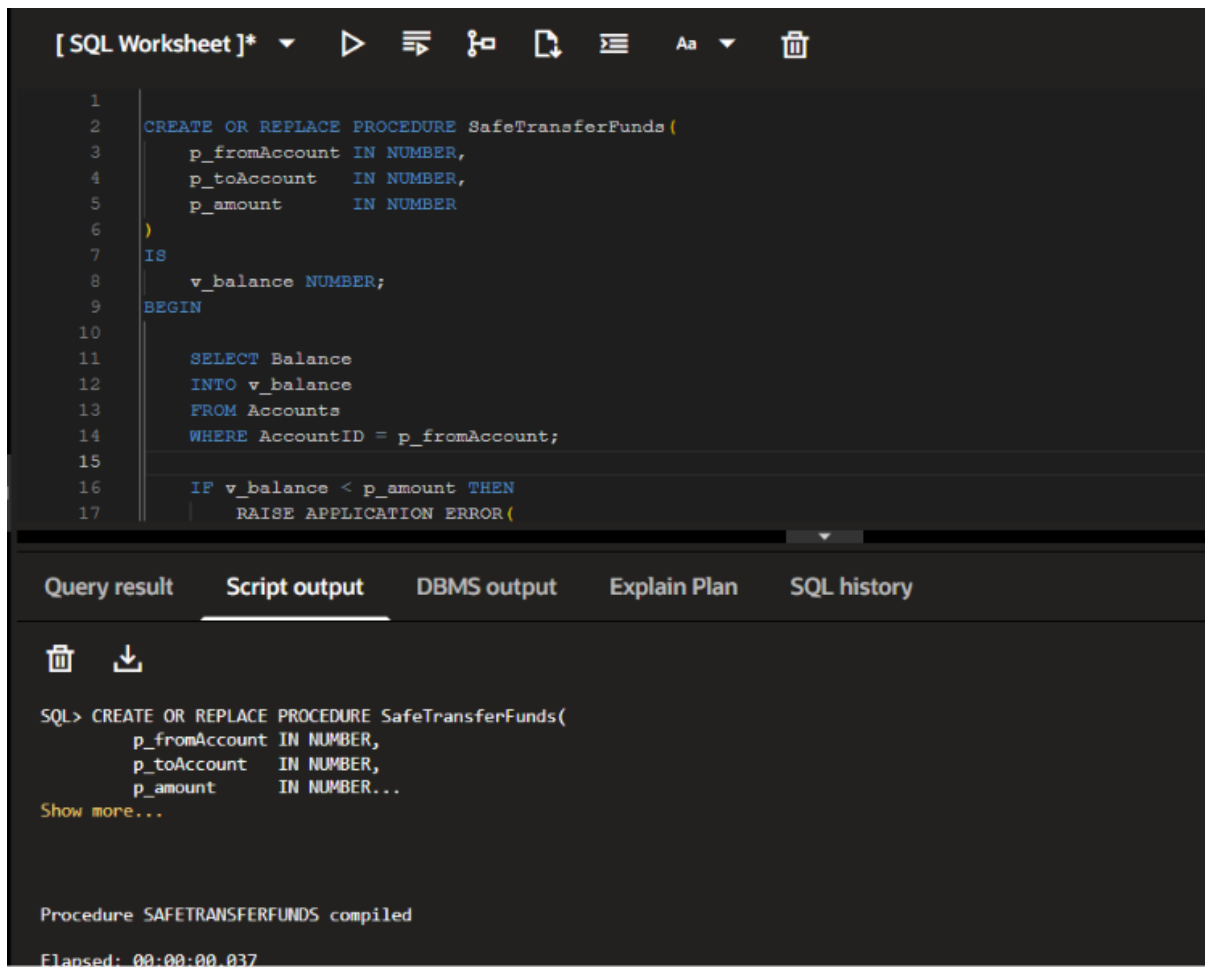
/* Test Case */

BEGIN
    AddNewCustomer(
        5,
        'David Miller',
        TO_DATE(
            '1995-06-10',
            'YYYY-MM-DD'
        ),
        8000
    );
END;
/

```

OUTPUT:

Scenario 1:



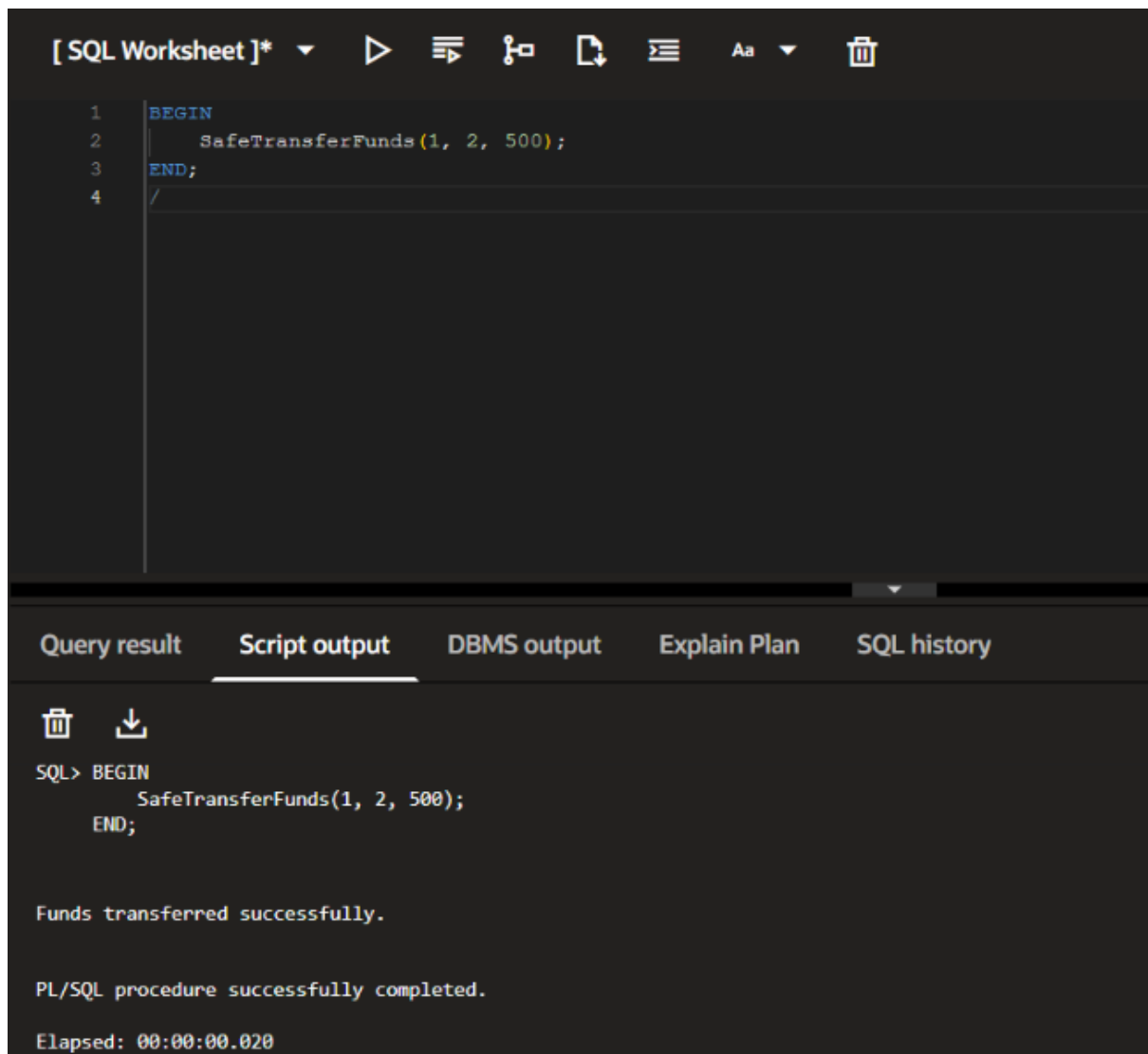
The screenshot displays an SQL IDE interface. The top toolbar includes icons for file operations, execution, and formatting. The main editor window shows the following SQL code:

```
1  
2 CREATE OR REPLACE PROCEDURE SafeTransferFunds(  
3     p_fromAccount IN NUMBER,  
4     p_toAccount   IN NUMBER,  
5     p_amount      IN NUMBER  
6 )  
7 IS  
8     v_balance NUMBER;  
9 BEGIN  
10  
11     SELECT Balance  
12     INTO v_balance  
13     FROM Accounts  
14     WHERE AccountID = p_fromAccount;  
15  
16     IF v_balance < p_amount THEN  
17         RAISE APPLICATION ERROR(
```

Below the editor, a tabbed interface shows the 'Script output' tab selected. It contains the following text:

```
SQL> CREATE OR REPLACE PROCEDURE SafeTransferFunds(  
      p_fromAccount IN NUMBER,  
      p_toAccount   IN NUMBER,  
      p_amount      IN NUMBER...  
Show more...
```

Below the script output, a status message indicates: 'Procedure SAFETRANSFERFUNDS compiled'. At the bottom, the execution time is shown as 'Elapsed: 00:00:00.037'.



The screenshot shows an SQL IDE interface. The top toolbar includes icons for running, saving, and other standard IDE functions. The main editor area contains a PL/SQL script with line numbers 1 through 4. The script starts with 'BEGIN', followed by an indented call to 'SafeTransferFunds(1, 2, 500);', then 'END;', and a forward slash on the final line. Below the editor, a tabbed interface shows 'Script output' as the active tab. This tab contains the executed script, a confirmation message 'Funds transferred successfully.', a completion message 'PL/SQL procedure successfully completed.', and an elapsed time of '00:00:00.020'.

```
[ SQL Worksheet ]*      Aa 
```

```
1 BEGIN  
2     SafeTransferFunds(1, 2, 500);  
3 END;  
4 /
```

Query result **Script output** DBMS output Explain Plan SQL history

SQL> BEGIN
 SafeTransferFunds(1, 2, 500);
 END;

Funds transferred successfully.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.020

Scenario 2:

[SQL Worksheet]*



Aa



```
1 CREATE OR REPLACE PROCEDURE UpdateSalary(  
2     p_employeeID IN NUMBER,  
3     p_percentage IN NUMBER  
4 )  
5 IS  
6     v_count NUMBER;  
7 BEGIN  
8  
9     SELECT COUNT(*)  
10    INTO v_count  
11    FROM Employees  
12    WHERE EmployeeID = p_employeeID;  
13  
14    IF v_count = 0 THEN  
15        RAISE_APPLICATION_ERROR(  
16            -20002,  
17            'Employee ID Not Found'
```

Query result

Script output

DBMS output

Explain Plan

SQL history



```
SQL> CREATE OR REPLACE PROCEDURE UpdateSalary(  
      p_employeeID IN NUMBER,  
      p_percentage IN NUMBER  
    )...  
Show more...
```

Procedure UPDATESALARY compiled

Elapsed: 00:00:00.033

[SQL Worksheet]* ▼



Aa ▼



```
1 BEGIN
2     UpdateSalary(1, 10);
3 END;
4 /
```

Query result

Script output

DBMS output

Explain Plan

SQL history



```
SQL> BEGIN
      UpdateSalary(1, 10);
      END;
```

Salary Updated Successfully.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.027

Scenario 3:

The screenshot displays a SQL IDE interface. The top toolbar includes icons for running, saving, and other standard SQL tool functions. The main editor area contains the following SQL code:

```
1 CREATE OR REPLACE PROCEDURE AddNewCustomer(  
2     p_customerID IN NUMBER,  
3     p_name        IN VARCHAR2,  
4     p_dob         IN DATE,  
5     p_balance     IN NUMBER  
6 )  
7 IS  
8 BEGIN  
9  
10    INSERT INTO Customers(  
11        CustomerID,  
12        Name,  
13        DOB,  
14        Balance,  
15        LastModified  
16    )  
17    VALUES (
```

Below the editor, a tabbed interface shows the 'Script output' tab selected. It contains the following text:

```
SQL> CREATE OR REPLACE PROCEDURE AddNewCustomer(  
      p_customerID IN NUMBER,  
      p_name        IN VARCHAR2,  
      p_dob         IN DATE,...  
Show more...
```

Below this, a status message reads: 'Procedure ADDNEWCUSTOMER compiled'. At the bottom, the execution time is shown as 'Elapsed: 00:00:00.030'.

[SQL Worksheet]*



Aa



```
1 BEGIN
2     AddNewCustomer(
3         5,
4         'David Miller',
5         TO_DATE('1995-06-10','YYYY-MM-DD'),
6         8000
7     );
8 END;
9 /
```

Query result

Script output

DBMS output

Explain Plan

SQL history



SQL> BEGIN

```
    AddNewCustomer(
        5,
        'David Miller',...
```

Show more...

Customer Added Successfully.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.011